

현대오토에버 코딩테스트 14 일 입문 워크북

알고리즘을 처음 배우는 사람을 위한 정답 선학습 과정

Java · Programmers 제출 형식 · 정답 선학습 방식

2026-07-14 버전

이 워크북의 사용법

핵심 원칙

처음 보는 유형은 오래 고민하지 않는다. 완성 정답을 먼저 이해하고, 빈칸 복원과 가까운 변형 문제로 같은 패턴을 반복한다.

1. 유형 판별 표를 정답과 함께 읽는다.
2. 완성 코드를 직접 한 번 입력하고 실행한다.
3. 코드를 가린 뒤 빈칸과 핵심 골격을 복원한다.
4. 변형 문제를 25 분 시도한 뒤 힌트를 순서대로 연다.
5. 정답과 비교하고 마지막 세 줄만 기록한다.

초기 1 주일의 권장 비중은 정답 학습 70%, 코드 복원 20%, 독립 풀이 10%다. Day 8 부터 독립 풀이 비중을 조금씩 늘린다.

막혔을 때의 규칙

- • 처음 보는 유형: 문제를 10 분 읽은 뒤 바로 정답 선학습으로 이동한다.
- • 배운 유형의 변형: 25 분 후 힌트 1, 15 분 후 힌트 2, 다시 15 분 후 정답을 본다.
- • 정답을 본 뒤에는 반드시 코드 전체를 달고 핵심 10~20 줄을 다시 쓴다.
- • 오류가 나면 알고리즘을 바꾸기 전에 경계, 방문 처리, 자료형, 초기값을 확인한다.

앞으로 사용할 호출 문구

예시

“4 일차 시작” -> Day 4 개념과 첫 문제 진행

“4 일차 힌트 2” -> Day 4 변형 문제의 두 번째 힌트

“4 일차 코드 봐줘” -> 작성한 Java 코드 검토

“7 일차 모의 시작” -> 1 주차 미니 모의 진행

14 일 고정 목차

일자	핵심 유형	대표 자료구조	목표
1	구현·시뮬레이션	배열·상태 변수	규칙을 코드로 옮기기
2	BFS	ArrayDeque·dist	같은 비용 최단거리
3	DFS	재귀·visited	연결 요소
4	정렬·그리디	정렬	정렬 기준 선택
5	우선순위 큐	PriorityQueue	동시 자원 수
6	다익스트라	PriorityQueue·dist	다른 비용 최단경로
7	1 주차 복습	복습	1 주차 유형 분류
8	기본 DP	1 차원 dp	상태와 점화식
9	배낭 DP	1 차원 배낭 dp	제한 자원 최적화
10	백트래킹	재귀·visited	순열·조합
11	누적합·이분 탐색	prefix·이분 탐색	합 질의와 답 탐색
12	비트마스크 BFS	mask·3 차원 visited	위치+수집 상태
13	다익스트라 전처리 + 비트마스크 DP	다익스트라·2^K DP	경유지 순서 최적화
14	최종 모의고사	종합	실전 2 문제

문제 출처와 저작권 안내

백준 문제 번호와 제목은 추가 연습 방향을 알려주는 참고 목록이다. 백준 원문은 수록하지 않았다. 워크북의 문제 설명, 예시, Java 코드는 학습용으로 새로 작성한 내용이다.

Day 1. 구현·시뮬레이션: 상태를 그대로 옮기기

예상 시간: 약 150 분

기준 백준: 14503 로봇 청소기 · 20055 컨베이어 벨트 위의 로봇

오늘 목표

- 문제에 적힌 처리 순서를 상태 변수로 옮긴다.
- 방향 배열과 경계 검사를 외운다.
- 알고리즘을 찾기 전에 정확한 구현이 필요한 문제를 구분한다.

유형 신호

명령을 순서대로 실행 / 회전·이동 / N 초 또는 N 번 반복 / 현재 위치·방향·배터리 같은 상태

1. 유형 판별 연습

처음에는 정답을 가리지 않는다. 문장과 정답 유형을 함께 읽고, 판단 근거에 밑줄을 긋는다.

문장	정답 유형	판단 근거
로봇에게 L, R, F 명령이 주어지고 최종 위치를 구한다.	구현·시뮬레이션	명령을 주어진 순서대로 실행하면 된다.
격자에서 벽을 피해 최소 이동 횟수를 구한다.	BFS	모든 이동 비용이 같고 최단 이동 횟수다.
각 칸의 통행료가 다를 때 최소 비용을 구한다.	다익스트라	가중치가 서로 다른 최단 비용 문제다.
회의를 최대한 많이 배정한다.	그리디	정렬 기준을 정해 앞에서부터 선택한다.
모든 체크포인트 방문 순서를 전부 확인한다.	백트래킹	순열 전체를 탐색한다.

2. 개념

구현 문제는 특별한 공식보다 문제의 규칙을 빠뜨리지 않고 코드로 옮기는 능력을 본다. 먼저 현재 상태를 적는다. 이 날의 상태는 행 r , 열 c , 방향 d , 이동 횟수처럼 몇 개의 변수로 표현된다.방향은 문자열로 관리하지 말고 0=위, 1=오른쪽, 2=아래, 3=왼쪽처럼 숫자로 관리한다. 그러면 왼쪽 회전은 $(d+3)\%4$, 오른쪽 회전은 $(d+1)\%4$ 로 계산할 수 있다.

이동 전에 반드시 새 좌표가 격자 안인지 확인하고, 장애물이 아닌지도 확인한다. 문제에서 동시 이동인지 순차 이동인지가 결과를 바꾸므로 한 턴의 처리 순서를 종이에 먼저 적는다.

3. 정답 선학습: 명령형 물류 로봇

0은 이동 가능, 1은 장애물인 격자와 시작 상태 [행, 열, 방향], 문자 명령열이 주어진다. F는 한 칸 전진, L과 R은 회전이다. 전진하려는 칸이 격자 밖이거나 장애물이면 위치는 바뀌지 않는다. 최종 [행, 열, 방향, 서로 다른 방문 칸 수]를 반환한다.

함수

```
public int[] solution(int[][] board, int[] start, String commands)
```

예시

```
board={{0,0,0},{0,1,0},{0,0,0}}, start={2,0,0}, commands="FFRFF" -> {0,2,1,5}
```

완성 정답 코드

```
import java.util.*;

class Solution {
    private static final int[] DR = {-1, 0, 1, 0};
    private static final int[] DC = {0, 1, 0, -1};

    public int[] solution(int[][] board, int[] start, String commands) {
        int rows = board.length;
        int cols = board[0].length;
        int r = start[0];
        int c = start[1];
        int direction = start[2];

        boolean[][] visited = new boolean[rows][cols];
        visited[r][c] = true;
        int visitedCount = 1;

        for (char command : commands.toCharArray()) {
            if (command == 'L') {
                direction = (direction + 3) % 4;
            } else if (command == 'R') {
                direction = (direction + 1) % 4;
            } else if (command == 'F') {
                int nr = r + DR[direction];
                int nc = c + DC[direction];
                if (nr < 0 || nr >= rows || nc < 0 || nc >= cols || board[nr][nc] == 1) continue;
                visited[nr][nc] = true;
                visitedCount++;
                r = nr;
                c = nc;
            }
        }

        return new int[] {r, c, direction, visitedCount};
    }
}
```

```

        direction = (direction + 1) % 4;
    } else if (command == 'F') {
        int nr = r + DR[direction];
        int nc = c + DC[direction];

        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) {
            continue;
        }
        if (board[nr][nc] == 1) {
            continue;
        }

        r = nr;
        c = nc;
        if (!visited[r][c]) {
            visited[r][c] = true;
            visitedCount++;
        }
    }
}

return new int[]{r, c, direction, visitedCount};
}
}

```

4. 오늘 외울 코드 카드

1. 방향 배열: dr={-1,0,1,0}, dc={0,1,0,-1}
2. 왼쪽 회전: (d+3)%4, 오른쪽 회전: (d+1)%4
3. 새 좌표 계산 -> 경계 검사 -> 장애물 검사 -> 상태 갱신

빈칸 복원

```

int nr = r + _____;
int nc = c + _____;

if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) {
    _____;
}
if (board[nr][nc] == 1) {
    _____;
}

```

5. 프로그래머스형 변형 문제: 창고 로봇 장거리 명령

0은 빈 칸, 1은 장애물인 창고에서 로봇이 움직인다. start는 [행, 열, 방향]이다. commands의 원소는 "LEFT", "RIGHT", "GO x" 중 하나다. GO x는 현재 방향으로 최대 x칸 전진하되, 경계나 장애물을 만나면 해당 명령을 즉시 종료한다. 최종 [행, 열, 방향, 실제 이동한 총 칸 수]를 반환하라.

제한사항

- 1 ≤ 행, 열 ≤ 100
- commands.length ≤ 10,000
- 1 ≤ x ≤ 100
- 시작 칸은 이동 가능

함수

```
public int[] solution(int[][] warehouse, int[] start, String[] commands)
```

예시

```
warehouse={{0,0,0,0},{0,1,1,0},{0,0,0,0}}, start={2,0,0}, commands={"GO 2","RIGHT","GO 3","RIGHT","GO 2"} -> {2,3,2,7}
```

6. 단계별 힌트

힌트 1

GO가 나오면 x번 반복하며 한 칸씩 검증한다.

힌트 2

장애물을 만나면 전체 명령 반복이 아니라 현재 GO 반복만 break 한다.

힌트 3

실제 위치가 바뀔 때만 movedCount를 증가시킨다.

7. 변형 문제 정답

정답 코드

```
import java.util.*;

class Solution {
    private static final int[] DR = {-1, 0, 1, 0};
    private static final int[] DC = {0, 1, 0, -1};

    public int[] solution(int[][] warehouse, int[] start, String[] commands) {
        int rows = warehouse.length;
        int cols = warehouse[0].length;
        int r = start[0];
        int c = start[1];
        int direction = start[2];
        int movedCount = 0;

        for (String command : commands) {
            if (command.equals("LEFT")) {
                direction = (direction + 3) % 4;
                continue;
            }
            if (command.equals("RIGHT")) {
                direction = (direction + 1) % 4;
                continue;
            }

            int steps = Integer.parseInt(command.substring(3));
            for (int step = 0; step < steps; step++) {
                int nr = r + DR[direction];
                int nc = c + DC[direction];

                if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) {
                    break;
                }
                if (warehouse[nr][nc] == 1) {
                    break;
                }

                r = nr;
                c = nc;
                movedCount++;
            }

            return new int[]{r, c, direction, movedCount};
        }
    }
}
```

8. 오늘의 복습

- ☐ 상태 변수 네 개를 말할 수 있는가?
- ☐ 회전 공식을 보지 않고 쓸 수 있는가?
- ☐ continue 와 break 의 차이를 이번 문제에서 설명할 수 있는가?

세 줄 기록

유형: _____ | 판단 단어: _____ | 외울 템플릿: _____

Day 2. BFS: 같은 비용의 최단 이동

예상 시간: 약 160 분

기준 백준: 2178 미로 탐색 · 14940 쉬운 최단거리

오늘 목표

- Queue 와 visited/dist 배열의 역할을 이해한다.
- 격자 최단 이동 횟수에서 BFS 를 선택한다.
- 방문 처리는 큐에 넣을 때 한다.

유형 신호

최소 이동 횟수 / 상하좌우 / 한 번 이동 비용이 모두 1 / 가장 가까운 위치

1. 유형 판별 연습

처음에는 정답을 가리지 않는다. 문장과 정답 유형을 함께 읽고, 판단 근거에 밑줄을 긋는다.

문장	정답 유형	판단 근거
모든 도로의 이동 시간이 1 이고 최단 이동 횟수를 구한다.	BFS	간선 비용이 모두 같다.
도로마다 시간이 다르고 최단 시간을 구한다.	다익스트라	가중치가 서로 다르다.
연결된 섬의 개수만 센다.	DFS/BFS	최단거리보다 연결 여부가 핵심이다.
문자열에서 괄호가 올바른지 확인한다.	스택	최근 열린 괄호부터 닫아야 한다.
모든 조합을 생성한다.	백트래킹	선택과 취소를 반복한다.

2. 개념

BFS 는 시작점과 가까운 상태부터 차례대로 확인한다. 한 번 이동할 때 드는 비용이 모두 같다면, 어떤 칸을 처음 방문한 순간의 거리가 그 칸까지의 최단거리다.

Queue 에는 앞으로 확인할 칸을 넣는다. dist[r][c]는 시작점에서 해당 칸까지의 이동 횟수다. 아직 방문하지 않은 칸은 -1 로 둔다.

방문 표시는 큐에서 꺼낼 때가 아니라 큐에 넣을 때 한다. 그래야 같은 칸이 여러 번 큐에 들어가는 것을 막을 수 있다.

3. 정답 선학습: 기본 격자 최단거리

1 은 이동 가능, 0 은 벽인 격자에서 좌측 상단에서 우측 하단까지 상하좌우로 이동하는 최소 이동 횟수를 반환한다. 도달할 수 없으면 -1 을 반환한다.

함수

public int solution(int[][] maps)

예시

maps={{1,1,0},{0,1,1},{1,1,1}} -> 4

완성 정답 코드

```
import java.util.*;

class Solution {
    private static final int[] DR = {-1, 0, 1, 0};
    private static final int[] DC = {0, 1, 0, -1};

    public int solution(int[][] maps) {
        int rows = maps.length;
        int cols = maps[0].length;
        int[][] dist = new int[rows][cols];
        for (int[] row : dist) {
            Arrays.fill(row, -1);
        }

        ArrayDeque<int[]> queue = new ArrayDeque<>();
        queue.offer(new int[]{0, 0});
        dist[0][0] = 0;

        while (!queue.isEmpty()) {
            int[] current = queue.poll();
            int r = current[0];
            int c = current[1];
```

```

        for (int d = 0; d < 4; d++) {
            int nr = r + DR[d];
            int nc = c + DC[d];

            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) {
                continue;
            }
            if (maps[nr][nc] == 0 || dist[nr][nc] != -1) {
                continue;
            }

            dist[nr][nc] = dist[r][c] + 1;
            queue.offer(new int[]{nr, nc});
        }
    }

    return dist[rows - 1][cols - 1];
}

```

4. 오늘 외울 코드 카드

1. ArrayDeque 를 Queue 로 사용한다.
2. dist 를 -1 로 채우면 방문 여부와 거리를 함께 관리할 수 있다.
3. dist[next] = dist[current] + 1 후 큐에 넣는다.

빈칸 복원

```

ArrayDeque<int[]> queue = new ArrayDeque<>();
queue._____(new int[]{startRow, startCol});
dist[startRow][startCol] = 0;

while (!queue._____) {
    int[] current = queue._____;
    // ...
    dist[nr][nc] = dist[r][c] + 1;
    queue._____(new int[]{nr, nc});
}

```

5. 프로그래머스형 변형 문제: 도시 내 긴급 출동

0은 도로, 1은 통행 불가 구역인 city가 주어진다. start와 target은 [행, 열]이다. 차량은 상하좌우로 한 칸씩 이동한다. target까지의 최소 이동 횟수를 반환하고, 도달할 수 없으면 -1을 반환하라.

제한사항

- 1 ≤ 행, 열 ≤ 200
- start와 target은 도로
- 각 이동의 비용은 1

함수

```
public int solution(int[][] city, int[] start, int[] target)
```

예시

city={{0,0,1,0,0},{1,0,1,0,1},{0,0,0,0,0},{0,1,1,1,0}}, start={0,0}, target={3,4} -> 7

6. 단계별 힌트

힌트 1

dist를 -1로 초기화한다.

힌트 2

start를 먼저 큐에 넣고 거리를 0으로 둔다.

힌트 3

target을 처음 발견했을 때 바로 반환해도 된다.

7. 변형 문제 정답

정답 코드

```

import java.util.*;

class Solution {
    private static final int[] DR = {-1, 0, 1, 0};
    private static final int[] DC = {0, 1, 0, -1};

    public int solution(int[][] city, int[] start, int[] target) {
        int rows = city.length;
        int cols = city[0].length;
        int[][] dist = new int[rows][cols];
        for (int[] row : dist) {
            Arrays.fill(row, -1);
        }

        ArrayDeque<int[]> queue = new ArrayDeque<>();
        queue.offer(new int[]{start[0], start[1]});
        dist[start[0]][start[1]] = 0;

        while (!queue.isEmpty()) {
            int[] current = queue.poll();
            int r = current[0];
            int c = current[1];

            if (r == target[0] && c == target[1]) {
                return dist[r][c];
            }

            for (int d = 0; d < 4; d++) {
                int nr = r + DR[d];
                int nc = c + DC[d];

                if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) {
                    continue;
                }
                if (city[nr][nc] == 1 || dist[nr][nc] != -1) {
                    continue;
                }

                dist[nr][nc] = dist[r][c] + 1;
                queue.offer(new int[]{nr, nc});
            }
        }

        return -1;
    }
}

```

8. 오늘의 복습

- ☐ 왜 BFS 는 처음 방문한 거리가 최단거리인가?
- ☐ 왜 방문 처리를 큐에 넣을 때 하는가?
- ☐ 이 문제에서 DFS 를 쓰면 어떤 점이 불편한가?

세 줄 기록

유형: _____ | 판단 단어: _____ | 외울 템플릿: _____

Day 3. DFS: 연결된 덩어리 찾기

예상 시간: 약 150 분

기준 백준: 2667 단지번호붙이기 · 1012 유기농 배추

오늘 목표

- 재귀 DFS 의 기본 골격을 외운다.
- 연결 요소의 개수와 크기를 센다.
- 최단거리 문제가 아닌 연결 탐색을 구분한다.

유형 신호

연결된 영역 / 섬·단지·클러스터 개수 / 모든 칸 방문 / 한 덩어리의 크기

1. 유형 판별 연습

처음에는 정답을 가리지 않는다. 문장과 정답 유형을 함께 읽고, 판단 근거에 밑줄을 긋는다.

문장	정답 유형	판단 근거
1로 연결된 영역의 개수와 크기를 센다.	DFS/BFS	각 연결 요소를 한 번씩 탐색한다.
가장 적은 이동 횟수를 구한다.	BFS	거리의 최소값이 핵심이다.
가중 그래프의 최소 비용을 구한다.	다익스트라	가중치 최단경로다.
N개 중 K개를 뽑는 모든 경우를 확인한다.	백트래킹	조합 탐색이다.
이전 결과로 현재 최댓값을 계산한다.	DP	부분 문제 결과를 재사용한다.

2. 개념

DFS 는 한 지점에서 갈 수 있는 곳까지 깊게 들어간 뒤 돌아오는 탐색이다. 연결 요소를 찾을 때는 아직 방문하지 않은 활성 칸을 발견할 때마다 DFS 를 한 번 시작한다.

DFS 한 번이 끝나면 하나의 연결된 덩어리를 모두 방문한 것이다. 따라서 DFS 를 시작한 횟수가 연결 요소의 개수다. 재귀 함수가 방문한 칸 수를 반환하도록 만들면 덩어리 크기도 계산할 수 있다.

재귀 함수의 첫 부분에는 경계, 대상 여부, 방문 여부 검사를 둔다. 이 세 가지 중 하나라도 실패하면 0 을 반환한다.

3. 정답 선학습: 활성 장비 구역

1 은 활성 장비, 0 은 빈 칸인 격자에서 상하좌우로 연결된 활성 구역의 개수와 가장 큰 구역의 크기를 반환한다.

함수

```
public int[] solution(int[][] grid)
```

예시

```
grid={{1,1,0},{0,1,0},{1,0,1}} -> {3,3}
```

완성 정답 코드

```
class Solution {
    private static final int[] DR = {-1, 0, 1, 0};
    private static final int[] DC = {0, 1, 0, -1};

    private int rows;
    private int cols;
    private int[][] grid;
    private boolean[][] visited;

    public int[] solution(int[][] grid) {
        this.grid = grid;
        rows = grid.length;
        cols = grid[0].length;
        visited = new boolean[rows][cols];

        int componentCount = 0;
        int maxSize = 0;

        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                if (grid[r][c] == 1 && !visited[r][c]) {
                    componentCount++;
                    maxSize = Math.max(maxSize, dfs(r, c));
                }
            }
        }
    }
}
```

```

    }

    return new int[]{componentCount, maxSize};
}

private int dfs(int r, int c) {
    visited[r][c] = true;
    int size = 1;

    for (int d = 0; d < 4; d++) {
        int nr = r + DR[d];
        int nc = c + DC[d];

        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) {
            continue;
        }
        if (grid[nr][nc] == 0 || visited[nr][nc]) {
            continue;
        }

        size += dfs(nr, nc);
    }
    return size;
}
}

```

4. 오늘 외울 코드 카드

1. 바깥 반복문에서 미방문 대상 칸을 발견하면 DFS 시작
2. DFS 시작 횟수 = 연결 요소 개수
3. size = 1 + 이웃 DFS 가 반환한 크기

빈칸 복원

```

if (grid[r][c] == 1 && !visited[r][c]) {
    componentCount++;
    int size = _____;
    maxSize = Math.max(maxSize, size);
}

private int dfs(int r, int c) {
    visited[r][c] = _____;
    int size = 1;
    // 이웃 크기를 size에 더한다.
}

```

5. 프로그래머스형 변형 문제: 서버 클러스터 크기

문자열 배열 network에서 문자 1은 동작 중인 서버, 0은 빈 칸이다. 상하좌우로 연결된 각 서버 클러스터의 크기를 오름차순 int[]로 반환하라. 서버가 하나도 없으면 빈 배열을 반환한다.

제한사항

- 1 ≤ 행, 열 ≤ 50
- 각 문자는 0 또는 1

함수

```
public int[] solution(String[] network)
```

예시

```
network={"11001","11001","00100","00111"} -> {2,4,4}
```

6. 단계별 힌트

힌트 1

각 미방문 1에서 DFS를 시작한다.

힌트 2

DFS가 반환한 크기를 List<Integer>에 넣는다.

힌트 3

마지막에 정렬하고 int[]로 바꾼다.

7. 변형 문제 정답

정답 코드

```
import java.util.*;

class Solution {
    private static final int[] DR = {-1, 0, 1, 0};
    private static final int[] DC = {0, 1, 0, -1};

    private String[] network;
    private boolean[][] visited;
    private int rows;
    private int cols;

    public int[] solution(String[] network) {
        this.network = network;
        rows = network.length;
        cols = network[0].length();
        visited = new boolean[rows][cols];

        List<Integer> sizes = new ArrayList<>();
        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                if (network[r].charAt(c) == '1' && !visited[r][c]) {
                    sizes.add(dfs(r, c));
                }
            }
        }

        Collections.sort(sizes);
        int[] answer = new int[sizes.size()];
        for (int i = 0; i < sizes.size(); i++) {
            answer[i] = sizes.get(i);
        }
        return answer;
    }

    private int dfs(int r, int c) {
        visited[r][c] = true;
        int size = 1;

        for (int d = 0; d < 4; d++) {
            int nr = r + DR[d];
            int nc = c + DC[d];
            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) {
                continue;
            }
            if (network[nr].charAt(nc) != '1' || visited[nr][nc]) {
                continue;
            }
            size += dfs(nr, nc);
        }
        return size;
    }
}
```

8. 오늘의 복습

- ☐ DFS 를 시작하는 위치는 어디인가?
- ☐ 방문 표시를 하지 않으면 어떤 문제가 생기는가?
- ☐ 재귀 깊이가 매우 깊을 때는 무엇을 고려해야 하는가?

세 줄 기록

유형: _____ | 판단 단어: _____ | 외울 템플릿: _____

Day 4. 정렬·그리다: 지금의 선택 기준 만들기

예상 시간: 약 160 분

기준 백준: 1931 회의실 배정 · 1744 수 묶기

오늘 목표

- 그리다 문제에서 정렬 기준을 찾는다.
- 회의실 배정의 종료 시간 기준을 외운다.
- 같은 종료 시간일 때의 보조 정렬 기준을 작성한다.

유형 신호

최대한 많이 선택 / 순서를 자유롭게 변경 / 앞에서부터 하나씩 결정 / 가장 빨리 끝나는 것

1. 유형 판별 연습

처음에는 정답을 가리지 않는다. 문장과 정답 유형을 함께 읽고, 판단 근거에 밑줄을 긋는다.

문장	정답 유형	판단 근거
겹치지 않는 회의를 최대한 많이 고른다.	그리다	빨리 끝나는 회의를 먼저 선택한다.
모든 방문 순서를 확인해 최소 비용을 구한다.	백트래킹/DP	하나의 정렬 기준만으로 보장되지 않는다.
각 구간 합을 수만 번 묻는다.	누적합	미리 합을 저장한다.
무게 제한 안에서 가치 합을 최대화한다.	배낭 DP	선택 조합의 최적값을 저장한다.
도로 비용이 모두 1 인 최단거리다.	BFS	같은 비용의 최단거리다.

2. 개념

그리다는 매 순간 하나를 선택하고 그 선택을 되돌리지 않는 방식이다. 핵심은 어떤 기준으로 정렬할지와 그 기준이 최적이라는 이유다.

회의를 최대한 많이 선택하려면 종료 시간이 빠른 회의부터 본다. 빨리 끝나는 회의를 선택하면 뒤에 남는 시간이 가장 많아진다. 종료 시간이 같다면 시작 시간이 빠른 순으로 정렬해도 된다.

코드보다 먼저 현재 선택한 마지막 회의의 종료 시각 lastEnd 를 적는다. 다음 회의의 시작 시각이 lastEnd 이상이면 선택하고 lastEnd 를 갱신한다.

3. 정답 선학습: 회의실 최대 배정

각 회의가 [시작, 종료]로 주어질 때 서로 겹치지 않게 선택할 수 있는 최대 회의 수를 반환한다. 종료 시각과 다음 회의 시작 시각이 같으면 연속 배정할 수 있다.

함수

```
public int solution(int[][] meetings)
```

예시

```
meetings={{1,4},{3,5},{0,6},{5,7},{5,9},{8,9}} -> 3
```

완성 정답 코드

```
import java.util.*;

class Solution {
    public int solution(int[][] meetings) {
        Arrays.sort(meetings, (a, b) -> {
            if (a[1] != b[1]) {
                return Integer.compare(a[1], b[1]);
            }
            return Integer.compare(a[0], b[0]);
        });

        int selected = 0;
        int lastEnd = Integer.MIN_VALUE;

        for (int[] meeting : meetings) {
            if (meeting[0] >= lastEnd) {
                selected++;
                lastEnd = meeting[1];
            }
        }
        return selected;
    }
}
```

}

4. 오늘 외울 코드 카드

1. 종료 시간 오름차순, 같으면 시작 시간 오름차순
2. start >= lastEnd 이면 선택
3. 선택 후 lastEnd = end

빈칸 복원

```
Arrays.sort(meetings, (a, b) -> {
    if (a[1] != b[1]) {
        return Integer.compare(_____, _____);
    }
    return Integer.compare(a[0], b[0]);
});

if (meeting[0] >= _____) {
    selected++;
    lastEnd = meeting[1];
}
```

5. 프로그래머스형 변형 문제: 차량 검사 예약

각 검사는 [시작, 종료]이며 검사 종료 후 cleanupTime 만큼 정리 시간이 필요하다. 한 검사대에서 겹치지 않게 수행할 수 있는 최대 검사 수를 반환하라. 검사 A 종료가 5 이고 cleanupTime 이 2 라면 다음 검사는 7 이상에 시작해야 한다.

제한사항

- 1 <= inspections.length <= 200,000
- 0 <= cleanupTime <= 1,000,000
- 시작 < 종료

함수

```
public int solution(int[][] inspections, int cleanupTime)
```

예시

```
inspections={{1,3},{3,5},{4,7},{7,8},{9,10}}, cleanupTime=1 -> 3
```

6. 단계별 힌트

힌트 1

정렬 기준은 여전히 종료 시간이 빠른 순서다.

힌트 2

선택한 검사의 실제 사용 종료 시각은 end + cleanupTime 이다.

힌트 3

시간 합이 int 범위를 넘을 수 있으면 long 을 사용한다.

7. 변형 문제 정답

정답 코드

```
import java.util.*;

class Solution {
    public int solution(int[][] inspections, int cleanupTime) {
        Arrays.sort(inspections, (a, b) -> {
            if (a[1] != b[1]) {
                return Integer.compare(a[1], b[1]);
            }
            return Integer.compare(a[0], b[0]);
        });

        int selected = 0;
        long availableAt = Long.MIN_VALUE;

        for (int[] inspection : inspections) {
            if (inspection[0] >= availableAt) {
                selected++;
                availableAt = inspection[1] + cleanupTime;
            }
        }

        return selected;
    }
}
```

```
        availableAt = (long) inspection[1] + cleanupTime;
    }
    }
    return selected;
}
```

8. 오늘의 복습

- ☐ 왜 시작 시간이 아니라 종료 시간을 기준으로 정렬하는가?
- ☐ 정렬 후 한 번만 순회하는 이유는 무엇인가?
- ☐ 그리디가 의심될 때 어떤 작은 반례를 만들어볼 것인가?

세 줄 기록

유형: _____ | 판단 단어: _____ | 외울 템플릿: _____

Day 5. 우선순위 큐: 가장 급한 후보부터 꺼내기

예상 시간: 약 160 분

기준 백준: 11000 강의실 배정 · 1202 보석 도둑

오늘 목표

- PriorityQueue의 최소 힙 기본 사용법을 외운다.
- 겹치는 구간의 최소 자원 수를 계산한다.
- 정렬과 우선순위 큐를 함께 사용한다.

유형 신호

동시에 필요한 최소 개수 / 현재 진행 중인 작업 / 가장 빨리 끝나는 작업 / 후보 중 최솟값·최댓값 반복 선택

1. 유형 판별 연습

처음에는 정답을 가리지 않는다. 문장과 정답 유형을 함께 읽고, 판단 근거에 밑줄을 긋는다.

문장	정답 유형	판단 근거
동시에 진행되는 작업을 처리할 최소 서버 수를 구한다.	정렬+우선순위 큐	현재 작업들의 종료 시각을 관리한다.
하나의 회의실에서 최대 회의 수를 구한다.	그리디	종료 시간 순으로 하나씩 선택한다.
가장 가까운 칸부터 탐색한다.	BFS	거리 계층을 큐로 처리한다.
최소 비용 정점을 반복해서 선택한다.	다익스트라	우선순위 큐로 현재 최소 dist를 꺼낸다.
모든 부분집합을 확인한다.	백트래킹/비트마스킹	선택 여부 전체를 탐색한다.

2. 개념

PriorityQueue는 지금 가장 작은 값 또는 가장 큰 값을 빠르게 꺼내는 자료구조다. Java의 기본 PriorityQueue<Integer>는 가장 작은 값이 먼저 나오는 최소 힙이다.

최소 자원 수 문제에서는 작업을 시작 시각 순으로 정렬한다. 우선순위 큐에는 현재 사용 중인 자원의 종료 시각을 넣는다. 새 작업 시작 전에 끝난 작업들을 큐에서 제거하고, 새 종료 시각을 넣는다.

큐의 크기는 현재 동시에 진행 중인 작업 수다. 순회 중 가장 큰 큐 크기가 필요한 최소 자원 수다.

3. 정답 선학습: 최소 작업실 수

작업 구간 [시작, 종료]가 주어진다. 한 작업실에서는 동시에 하나의 작업만 수행할 수 있다. 모든 작업을 수행하기 위한 최소 작업실 수를 반환한다. 종료 시각과 시작 시각이 같으면 같은 작업실을 재사용할 수 있다.

함수

```
public int solution(int[][] sessions)
```

예시

```
sessions={{1,4},{2,5},{4,7},{6,8}} -> 2
```

완성 정답 코드

```
import java.util.*;

class Solution {
    public int solution(int[][] sessions) {
        Arrays.sort(sessions, Comparator.comparingInt(a -> a[0]));
        PriorityQueue<Integer> endTimes = new PriorityQueue<>();
        int answer = 0;

        for (int[] session : sessions) {
            while (!endTimes.isEmpty() && endTimes.peek() <= session[0]) {
                endTimes.poll();
            }
            endTimes.offer(session[1]);
            answer = Math.max(answer, endTimes.size());
        }
        return answer;
    }
}
```

4. 오늘 외울 코드 카드

1. 시작 시간으로 정렬
2. peek() <= start 인 종료 시각을 제거
3. 새 end 를 offer 하고 최대 size 갱신

빈칸 복원

```
PriorityQueue<Integer> endTimes = new _____<>();

while (!endTimes.isEmpty() && endTimes.______() <= session[0]) {
    endTimes.______();
}
endTimes.______(session[1]);
answer = Math.max(answer, endTimes.______());
```

5. 프로그래머스형 변형 문제: 충전기 최소 수량

각 차량 충전 예약은 [시작, 종료]다. 충전 종료 후 turnaround 만큼 냉각 시간이 지나야 충전기를 다시 사용할 수 있다. 모든 예약을 처리하기 위한 최소 충전기 수를 반환하라.

제한사항

- • 1 <= 예약 수 <= 200,000
- • 0 <= turnaround <= 1,000,000
- • 시간은 int 범위

함수

```
public int solution(int[][] chargingSessions, int turnaround)
```

예시

```
chargingSessions={{1,4},{2,5},{4,7},{6,8}}, turnaround=1 -> 3
```

6. 단계별 힌트

힌트 1

실제 충전기 반환 시각은 종료+turnaround 다.

힌트 2

반환 시각이 새 시작 시각 이하인 항목은 모두 큐에서 제거한다.

힌트 3

반환 시각 합은 long 으로 계산한다.

7. 변형 문제 정답

정답 코드

```
import java.util.*;

class Solution {
    public int solution(int[][] chargingSessions, int turnaround) {
        Arrays.sort(chargingSessions,
            Comparator.comparingInt(a -> a[0]));

        PriorityQueue<Long> releaseTimes = new PriorityQueue<>();
        int answer = 0;

        for (int[] session : chargingSessions) {
            while (!releaseTimes.isEmpty()
                && releaseTimes.peek() <= session[0]) {
                releaseTimes.poll();
            }

            releaseTimes.offer((long) session[1] + turnaround);
            answer = Math.max(answer, releaseTimes.size());
        }
        return answer;
    }
}
```


8. 오늘의 복습

- ☐ 최소 힙의 peek 는 무엇인가?
- ☐ 종료된 작업을 왜 큐에서 제거하는가?
- ☐ 회의실 최대 개수 문제와 최소 작업실 수 문제의 차이는 무엇인가?

세 줄 기록

유형: _____ | 판단 단어: _____ | 외울 템플릿: _____

Day 6. 다익스트라: 비용이 다른 최단경로

예상 시간: 약 180 분

기준 백준: 1916 최소비용 구하기 · 4485 녹색 옷 입은 애가 젤다지?

오늘 목표

- BFS 와 다익스트라의 선택 기준을 구분한다.
- dist 배열과 PriorityQueue 갱신 패턴을 외운다.
- 오래된 우선순위 큐 항목을 건너뛴다.

유형 신호

최소 비용 / 칸·간선마다 비용이 다름 / 비용은 음수가 아님 / PriorityQueue + dist

1. 유형 판별 연습

처음에는 정답을 가리지 않는다. 문장과 정답 유형을 함께 읽고, 판단 근거에 밑줄을 긋는다.

문장	정답 유형	판단 근거
모든 이동 비용이 1 인 최단거리다.	BFS	가중치가 동일하다.
칸마다 배터리 소비량이 다르다.	다익스트라	가중치가 서로 다르다.
음수 간선이 있는 최단거리다.	다익스트라 사용 불가	다익스트라의 전제가 깨진다.
연결 요소 개수만 구한다.	DFS/BFS	최단비용이 필요하지 않다.
경유지 순서를 모두 확인한다.	백트래킹/비트마스크 DP	순서 최적화가 추가로 필요하다.

2. 개념

다익스트라는 아직 확정되지 않은 정점 중 현재까지 비용이 가장 작은 정점을 먼저 처리한다. 비용이 음수가 없을 때 사용할 수 있다.

dist[r][c]는 시작점에서 해당 칸까지 발견한 최소 비용이다. 새 비용이 더 작으면 dist 를 갱신하고 우선순위 큐에 새 상태를 넣는다.

우선순위 큐에는 같은 칸의 오래된 비용이 남을 수 있다. 큐에서 꺼낸 비용이 dist 와 다르면 이미 더 좋은 경로가 발견된 것이므로 continue 한다.

3. 정답 선학습: 격자 최소 배터리 비용

각 칸에 진입할 때 소비되는 양의 배터리 비용이 주어진다. 좌측 상단에서 우측 하단까지 상하좌우로 이동할 때의 최소 총비용을 반환한다. 시작 칸과 도착 칸 비용도 포함한다.

함수

```
public long solution(int[][] cost)
```

예시

```
cost={{1,5,1},{2,2,2},{9,1,1}} -> 7
```

완성 정답 코드

```
import java.util.*;

class Solution {
    private static final long INF = Long.MAX_VALUE / 4;
    private static final int[] DR = {-1, 0, 1, 0};
    private static final int[] DC = {0, 1, 0, -1};

    private static class Node implements Comparable<Node> {
        int r;
        int c;
        long cost;

        Node(int r, int c, long cost) {
            this.r = r;
            this.c = c;
            this.cost = cost;
        }

        public int compareTo(Node other) {
            return Long.compare(this.cost, other.cost);
        }
    }
}
```

```

    }
}

public long solution(int[][] cost) {
    int rows = cost.length;
    int cols = cost[0].length;
    long[][] dist = new long[rows][cols];
    for (long[] row : dist) {
        Arrays.fill(row, INF);
    }

    PriorityQueue<Node> pq = new PriorityQueue<>();
    dist[0][0] = cost[0][0];
    pq.offer(new Node(0, 0, dist[0][0]));

    while (!pq.isEmpty()) {
        Node current = pq.poll();
        if (current.cost != dist[current.r][current.c]) {
            continue;
        }

        for (int d = 0; d < 4; d++) {
            int nr = current.r + DR[d];
            int nc = current.c + DC[d];
            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) {
                continue;
            }

            long nextCost = current.cost + cost[nr][nc];
            if (nextCost < dist[nr][nc]) {
                dist[nr][nc] = nextCost;
                pq.offer(new Node(nr, nc, nextCost));
            }
        }
    }

    return dist[rows - 1][cols - 1];
}

```

4. 오늘 외울 코드 카드

1. dist를 INF로 초기화
2. 새 비용 < dist[next]이면 갱신 후 offer
3. poll 한 비용 != dist[current]이면 오래된 항목이므로 continue

빈칸 복원

```

long nextCost = current.cost + cost[nr][nc];
if (nextCost < _____) {
    dist[nr][nc] = _____;
    pq.offer(new Node(nr, nc, _____));
}

if (current.cost != dist[current.r][current.c]) {
    _____;
}

```

5. 프로그래머스형 변형 문제: 통행 불가 구역이 있는 최소 비용 경로

city의 -1은 통행 불가, 양수는 해당 칸에 진입할 때 드는 비용이다. start에서 target까지 상하좌우로 이동하는 최소 총비용을 반환하라. 시작 칸 비용도 포함하며 도달할 수 없으면 -1을 반환한다.

제한사항

- 1 ≤ 행, 열 ≤ 200
- 이동 가능 칸 비용 ≤ 1,000,000,000
- start와 target은 이동 가능

함수

```
public long solution(int[][] city, int[] start, int[] target)
```

예시

```
city={{1,3,-1,2},{2,1,4,2},{-1,2,1,1}}, start={0,0}, target={2,3} -> 8
```

6. 단계별 힌트

힌트 1

dist[start]를 city[start]로 둔다.

힌트 2

-1 칸은 이웃 후보에서 제외한다.

힌트 3

최소 비용 합은 long 으로 계산한다.

7. 변형 문제 정답

정답 코드

```
import java.util.*;

class Solution {
    private static final long INF = Long.MAX_VALUE / 4;
    private static final int[] DR = {-1, 0, 1, 0};
    private static final int[] DC = {0, 1, 0, -1};

    private static class Node implements Comparable<Node> {
        int r;
        int c;
        long cost;

        Node(int r, int c, long cost) {
            this.r = r;
            this.c = c;
            this.cost = cost;
        }

        public int compareTo(Node other) {
            return Long.compare(cost, other.cost);
        }
    }

    public long solution(int[][] city, int[] start, int[] target) {
        int rows = city.length;
        int cols = city[0].length;
        long[][] dist = new long[rows][cols];
        for (long[] row : dist) {
            Arrays.fill(row, INF);
        }

        PriorityQueue<Node> pq = new PriorityQueue<>();
        dist[start[0]][start[1]] = city[start[0]][start[1]];
        pq.offer(new Node(start[0], start[1], dist[start[0]][start[1]]));

        while (!pq.isEmpty()) {
            Node current = pq.poll();
            if (current.cost != dist[current.r][current.c]) {
                continue;
            }
            if (current.r == target[0] && current.c == target[1]) {
                return current.cost;
            }

            for (int d = 0; d < 4; d++) {
                int nr = current.r + DR[d];
                int nc = current.c + DC[d];
                if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) {
                    continue;
                }
                if (city[nr][nc] == -1) {
                    continue;
                }

                long nextCost = current.cost + city[nr][nc];
                if (nextCost < dist[nr][nc]) {
                    dist[nr][nc] = nextCost;
                    pq.offer(new Node(nr, nc, nextCost));
                }
            }
        }
        return -1L;
    }
}
```

8. 오늘의 복습

- ☐ BFS 대신 다익스트라를 선택한 단어는 무엇인가?
- ☐ 오래된 PQ 항목은 왜 생기는가?
- ☐ 왜 int 가 아니라 long 을 사용해야 하는가?

세 줄 기록

유형: _____ | 판단 단어: _____ | 외울 템플릿: _____

Day 7. 1 주차 복습: 유형을 먼저 맞히는 미니 모의

예상 시간: 180 분

기준 백준: 7576 토마토 · 11000 강의실 배정 · 4485 녹색 옷 입은 애가 젤다지?

오늘 목표

- 구현, BFS, DFS, 그리디, PQ, 다익스트라를 문장만 보고 구분한다.
- 정답 코드를 보지 않고 한 문제를 40 분 시도한다.
- 막히면 단계별 힌트를 사용한다.

유형 신호

오늘은 새로운 알고리즘 없음 / 분류 -> 템플릿 선택 -> 구현

1. 유형 판별 연습

처음에는 정답을 가리지 않는다. 문장과 정답 유형을 함께 읽고, 판단 근거에 밑줄을 긋는다.

문장	정답 유형	판단 근거
여러 충전소에서 각 도로까지의 가장 가까운 거리를 동시에 구한다.	다중 시작점 BFS	모든 충전소를 처음 큐에 넣는다.
검사 예약을 처리할 최소 장비 수를 구한다.	정렬+우선순위 큐	진행 중인 종료 시각을 관리한다.
통행료가 다른 도시의 최소 비용을 구한다.	다익스트라	가중치가 다르다.
연결된 장애물 구역의 개수를 센다.	DFS/BFS	연결 요소 탐색이다.
회전을 포함한 명령을 그대로 실행한다.	구현·시뮬레이션	주어진 규칙 순서대로 상태를 바꾼다.

2. 개념

오늘은 문제를 읽자마자 코드를 쓰지 않는다. 먼저 유형 이름 하나와 사용할 자료구조 하나를 적는다. 그 뒤 기존 템플릿을 꺼낸다.

처음 20 분 동안 접근이 전혀 보이지 않으면 힌트 1 을 본다. 15 분 뒤에도 구현을 시작하지 못하면 힌트 2 를 본다. 이 단계에서는 오래 버티는 것보다 올바른 패턴을 반복해서 익히는 것이 중요하다.

문제 A는 다중 시작점 BFS 다. 시작점이 여러 개라는 점만 다르고 BFS 템플릿은 같다. 모든 시작점을 거리 0 으로 큐에 넣으면 가장 가까운 시작점에서부터 동시에 퍼진다.

3. 정답 선학습: 가장 먼 도로의 충전소 거리

city에서 -1 은 벽, 0 은 도로, 1 은 충전소다. 각 도로 칸에서 가장 가까운 충전소까지의 최단 이동 횟수 중 최댓값을 반환한다. 충전소가 없거나 어떤 도로가 충전소에 도달할 수 없으면 -1 을 반환한다.

함수

public int solution(int[][] city)

예시

city={{0,0,-1},{0,-1,0,0},{0,0,0,0}} -> 8

완성 정답 코드

```
import java.util.*;

class Solution {
    private static final int[] DR = {-1, 0, 1, 0};
    private static final int[] DC = {0, 1, 0, -1};

    public int solution(int[][] city) {
        int rows = city.length;
        int cols = city[0].length;
        int[][] dist = new int[rows][cols];
        for (int[] row : dist) {
            Arrays.fill(row, -1);
        }

        ArrayDeque<int[]> queue = new ArrayDeque<>();
        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                if (city[r][c] == 1) {
                    dist[r][c] = 0;
                    queue.offer(new int[]{r, c});
                }
            }
        }

        while (!queue.isEmpty()) {
            int[] cur = queue.poll();
            int r = cur[0], c = cur[1];
            for (int d = 0; d < 4; d++) {
                int nr = r + DR[d], nc = c + DC[d];
                if (nr < 0 || nr > rows - 1 || nc < 0 || nc > cols - 1 || city[nr][nc] == -1 || dist[nr][nc] != -1) continue;
                dist[nr][nc] = dist[r][c] + 1;
                queue.offer(new int[]{nr, nc});
            }
        }

        int answer = -1;
        for (int[] row : dist) {
            for (int val : row) {
                answer = Math.max(answer, val);
            }
        }

        return answer;
    }
}
```

```

    }
}
if (queue.isEmpty()) {
    return -1;
}

while (!queue.isEmpty()) {
    int[] current = queue.poll();
    for (int d = 0; d < 4; d++) {
        int nr = current[0] + DR[d];
        int nc = current[1] + DC[d];
        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) {
            continue;
        }
        if (city[nr][nc] == -1 || dist[nr][nc] != -1) {
            continue;
        }
        dist[nr][nc] = dist[current[0]][current[1]] + 1;
        queue.offer(new int[]{nr, nc});
    }
}

int answer = 0;
for (int r = 0; r < rows; r++) {
    for (int c = 0; c < cols; c++) {
        if (city[r][c] == 0) {
            if (dist[r][c] == -1) {
                return -1;
            }
            answer = Math.max(answer, dist[r][c]);
        }
    }
}
return answer;
}
}

```

4. 오늘 외울 코드 카드

1. 시작점이 여러 개면 모두 거리 0 으로 큐에 넣는다.
2. 그 이후 BFS 코드는 한 시작점일 때와 같다.
3. 마지막에 도달하지 못한 도로가 있는지 확인한다.

빈칸 복원

```

for (모든 칸) {
    if (city[r][c] == 1) {
        dist[r][c] = ____;
        queue.offer(new int[]{r, c});
    }
}

// 이후는 일반 BFS 와 동일

```

5. 프로그래머스형 변형 문제: 1 주차 미니 모의 B - 냉각 시간이 있는 충전 예약

각 예약은 [시작, 종료]다. 종료 후 cooldown 동안 충전기를 사용할 수 없다. 모든 예약을 처리할 최소 충전기 수를 반환하라. Day 5 코드를 보지 않고 먼저 35 분 시도한다.

제한사항

- 1 ≤ 예약 수 ≤ 300,000
- 0 ≤ cooldown ≤ 1,000,000

함수

```
public int solution(int[][] reservations, int cooldown)
```

예시

```
reservations={{0,5},{1,2},{3,7},{6,9}}, cooldown=1 -> 2
```

6. 단계별 힌트

힌트 1

예약을 시작 시간 순으로 정렬한다.

힌트 2

PriorityQueue 에는 종료+cooldown 을 넣는다.

힌트 3

새 시작 전에 반환된 충전기는 모두 제거한다.

7. 변형 문제 정답

정답 코드

```
import java.util.*;

class Solution {
    public int solution(int[][] reservations, int cooldown) {
        Arrays.sort(reservations,
            Comparator.comparingInt(a -> a[0]));
        PriorityQueue<Long> releaseTimes = new PriorityQueue<>();
        int answer = 0;

        for (int[] reservation : reservations) {
            while (!releaseTimes.isEmpty()
                && releaseTimes.peek() <= reservation[0]) {
                releaseTimes.poll();
            }
            releaseTimes.offer((long) reservation[1] + cooldown);
            answer = Math.max(answer, releaseTimes.size());
        }
        return answer;
    }
}
```

8. 오늘의 복습

- ☐ 문제 문장에서 유형을 고르는 데 걸린 시간은?
- ☐ BFS 와 다익스트라를 구분하는 한 문장은?
- ☐ 다음 주에 다시 써야 할 템플릿 두 개는?

세 줄 기록

유형: _____ | 판단 단어: _____ | 외울 템플릿: _____

Day 8. 기본 DP: 이전 답으로 현재 답 만들기

예상 시간: 약 170 분

기준 백준: 1463 1로 만들기 · 1149 RGB 거리

오늘 목표

- $dp[i]$ 의 의미를 한 문장으로 정의한다.
- 초기값과 점화식을 분리해서 작성한다.
- 같은 계산이 반복될 때 DP를 의심한다.

유형 신호

최댓값·최솟값 / 이전 단계의 결과 / 같은 부분 문제가 반복 / 선택하거나 건너뛰기

1. 유형 판별 연습

처음에는 정답을 가리지 않는다. 문장과 정답 유형을 함께 읽고, 판단 근거에 밑줄을 긋는다.

문장	정답 유형	판단 근거
인접한 두 항목을 동시에 고를 수 없을 때 최대 합을 구한다.	DP	i 번째를 고르는 경우와 건너뛰는 경우를 비교한다.
가장 빨리 끝나는 회의의 반복 선택한다.	그리디	현재 선택 기준으로 바로 결정한다.
모든 순열을 생성한다.	백트래킹	상태를 선택하고 되돌린다.
최소 이동 횟수를 구한다.	BFS	같은 비용의 최단거리다.
가중치가 다른 최소 경로다.	다익스트라	dist를 갱신한다.

2. 개념

DP는 큰 문제의 답을 더 작은 문제의 답으로 만든다. 먼저 $dp[i]$ 가 무엇인지 한 문장으로 적는다. 이 문장이 모호하면 점화식도 흔들린다.

인접한 보상을 동시에 선택할 수 없는 문제에서 $dp[i]$ 를 0번부터 i 번까지 고려한 최대 보상이라고 정의한다. i 를 고르면 $dp[i-2] + \text{reward}[i]$, 고르지 않으면 $dp[i-1]$ 이다.

초기값은 배열 길이가 1인 경우까지 안전하게 처리해야 한다. 값의 합이 커질 수 있으므로 long 사용 여부도 확인한다.

3. 정답 선학습: 인접하지 않은 보상 최대화

$\text{reward}[i]$ 는 i 번째 작업의 보상이다. 서로 인접한 작업은 동시에 선택할 수 없다. 얻을 수 있는 최대 보상 합을 반환한다.

함수

```
public long solution(int[] reward)
```

예시

```
reward={5,1,2,10,6,2} -> 17
```

완성 정답 코드

```
class Solution {
    public long solution(int[] reward) {
        if (reward.length == 0) {
            return 0L;
        }
        if (reward.length == 1) {
            return reward[0];
        }

        long[] dp = new long[reward.length];
        dp[0] = reward[0];
        dp[1] = Math.max(reward[0], reward[1]);

        for (int i = 2; i < reward.length; i++) {
            long skip = dp[i - 1];
            long take = dp[i - 2] + reward[i];
            dp[i] = Math.max(skip, take);
        }
        return dp[reward.length - 1];
    }
}
```

4. 오늘 외울 코드 카드

1. $dp[i] = 0..i$ 까지 고려한 최대값
2. $skip=dp[i-1]$, $take=dp[i-2]+value[i]$
3. $dp[i]=\max(skip, take)$

빈칸 복원

```
long skip = _____;
long take = _____ + reward[i];
dp[i] = Math._____(skip, take);
```

5. 프로그래머스형 변형 문제: 체크포인트 최소 에너지

로봇은 0 번 체크포인트에서 출발해 마지막 체크포인트까지 이동한다. 한 번에 1 칸 또는 2 칸 앞으로 이동할 수 있고, 방문한 체크포인트의 energy 비용을 지불한다. 시작 체크포인트 비용도 지불한다. 마지막에 도달하는 최소 총에너지를 반환하라.

제한사항

- $1 \leq \text{energy.length} \leq 200,000$
- $1 \leq \text{energy}[i] \leq 1,000,000,000$

함수

```
public long solution(int[] energy)
```

예시

```
energy={4,2,7,1,3} -> 10
```

6. 단계별 힌트

힌트 1

$dp[i]$ 를 i 에 도착하는 최소 비용으로 정의한다.

힌트 2

i 는 $i-1$ 또는 $i-2$ 에서 올 수 있다.

힌트 3

$dp[i]=\text{energy}[i]+\min(dp[i-1], dp[i-2])$ 다.

7. 변형 문제 정답

정답 코드

```
class Solution {
    public long solution(int[] energy) {
        if (energy.length == 1) {
            return energy[0];
        }

        long[] dp = new long[energy.length];
        dp[0] = energy[0];
        dp[1] = (long) energy[0] + energy[1];

        for (int i = 2; i < energy.length; i++) {
            dp[i] = energy[i] + Math.min(dp[i - 1], dp[i - 2]);
        }
        return dp[energy.length - 1];
    }
}
```

8. 오늘의 복습

- ☐ $dp[i]$ 를 한 문장으로 정의해보라.
- ☐ 초기값 두 개가 필요한 이유는?
- ☐ 그리디와 DP의 차이를 현재 선택을 되돌릴 수 있는지로 설명해보라.

세 줄 기록

유형: _____ | 판단 단어: _____ | 외울 템플릿: _____

Day 9. 배낭 DP: 한정된 자원으로 최대 가치

예상 시간: 약 180 분

기준 백준: 12865 평범한 배낭 · 7579 앱

오늘 목표

- 0/1 배낭 문제의 상태를 이해한다.
- 용량을 뒤에서 앞으로 순회하는 이유를 외운다.
- 무게·시간·비용 제한 문제를 배낭으로 인식한다.

유형 신호

각 항목을 한 번만 선택 / 총 무게·시간·비용 제한 / 가치·효과 최대화 / 선택 조합

1. 유형 판별 연습

처음에는 정답을 가리지 않는다. 문장과 정답 유형을 함께 읽고, 판단 근거에 밑줄을 긋는다.

문장	정답 유형	판단 근거
시간 제한 안에서 테스트 효과 합을 최대화한다.	0/1 배낭 DP	각 테스트를 한 번 선택하고 총 시간이 제한된다.
가장 빨리 끝나는 작업을 선택한다.	그리디	정렬 기준으로 즉시 결정한다.
같은 물건을 여러 번 선택할 수 있다.	완전 배낭	용량 순회 방향이 달라진다.
경유지 순서를 모두 확인한다.	백트래킹/비트마스킹 DP	순서 상태가 필요하다.
구간 합 질문이 반복된다.	누적합	합을 미리 계산한다.

2. 개념

0/1 배낭은 각 물건을 선택하거나 선택하지 않는 문제다. $dp[c]$ 를 용량 c 이하에서 얻을 수 있는 최대 가치로 정의한다.

현재 물건의 무게가 w , 가치가 v 라면 $dp[c] = \max(dp[c], dp[c-w]+v)$ 다. 같은 물건을 한 번만 사용해야 하므로 c 를 capacity 에서 w 까지 감소시키며 순회한다.

앞에서 뒤로 순회하면 방금 갱신한 dp 를 다시 사용해 같은 물건을 여러 번 넣는 효과가 생긴다. 이 순회 방향은 반드시 암기한다.

3. 정답 선학습: 차량 옵션 선택

capacity 이하의 총 무게로 옵션을 선택한다. $items[i]$ 는 [무게, 가치]이며 각 옵션은 최대 한 번 선택할 수 있다. 최대 가치 합을 반환한다.

함수

```
public int solution(int capacity, int[][] items)
```

예시

capacity=7, items={{3,6},{4,8},{2,4},{5,9}} -> 14

완성 정답 코드

```
class Solution {
    public int solution(int capacity, int[][] items) {
        int[] dp = new int[capacity + 1];

        for (int[] item : items) {
            int weight = item[0];
            int value = item[1];

            for (int c = capacity; c >= weight; c--) {
                dp[c] = Math.max(dp[c], dp[c - weight] + value);
            }
        }
        return dp[capacity];
    }
}
```

4. 오늘 외울 코드 카드

1. $dp[c]$ = 용량 c 이하에서의 최대 가치
2. 각 물건마다 c 를 capacity 부터 weight 까지 감소

3. $dp[c] = \max(dp[c], dp[c-weight]+value)$

빈칸 복원

```
for (int[] item : items) {
    int weight = item[0];
    int value = item[1];

    for (int c = capacity; c >= _____; c--) {
        dp[c] = Math.max(dp[c], dp[c - _____] + _____);
    }
}
```

5. 프로그래머스형 변형 문제: 회귀 테스트 선택

전체 실행 시간 `timeLimit` 안에서 회귀 테스트를 선택한다. `tests[i]`는 [실행 시간, 결함 탐지 점수]이며 각 테스트는 한 번만 선택할 수 있다. 최대 탐지 점수를 반환하라.

제한사항

- • $1 \leq \text{timeLimit} \leq 100,000$
- • $1 \leq \text{tests.length} \leq 2,000$
- • 시간과 점수는 양수

함수

```
public int solution(int timeLimit, int[][] tests)
```

예시

`timeLimit=8, tests={{3,5},{4,7},{5,9},{2,4}}` -> 14

6. 단계별 힌트

힌트 1

시간을 배낭 용량으로 본다.

힌트 2

각 테스트를 한 번만 사용할 수 있으므로 역순 순회한다.

힌트 3

정답은 `dp[timeLimit]`이다.

7. 변형 문제 정답

정답 코드

```
class Solution {
    public int solution(int timeLimit, int[][] tests) {
        int[] dp = new int[timeLimit + 1];

        for (int[] test : tests) {
            int time = test[0];
            int score = test[1];
            for (int used = timeLimit; used >= time; used--) {
                dp[used] = Math.max(
                    dp[used],
                    dp[used - time] + score
                );
            }
        }
        return dp[timeLimit];
    }
}
```

8. 오늘의 복습

- ☐ 왜 용량을 역순으로 순회하는가?
- ☐ 같은 물건을 여러 번 선택할 수 있다면 순회 방향은?
- ☐ `dp[c]`의 의미를 정확히 말할 수 있는가?

세 줄 기록

유형: _____ | 판단 단어: _____ | 외울 템플릿: _____

Day 10. 백트래킹: 선택하고 되돌리기

예상 시간: 약 180 분

기준 백준: 15649 N과 M (1) · 15686 치킨 배달

오늘 목표

- 순열과 조합을 구분한다.
- 선택-재귀-선택 취소 구조를 외운다.
- 작은 K에서 완전탐색이 가능한지 판단한다.

유형 신호

모든 순서 / N 개 중 K 개 선택 / 경우를 전부 확인 / 선택 후 되돌리기

1. 유형 판별 연습

처음에는 정답을 가리지 않는다. 문장과 정답 유형을 함께 읽고, 판단 근거에 밑줄을 긋는다.

문장	정답 유형	판단 근거
체크포인트 방문 순서가 결과에 영향을 준다.	순열 백트래킹	순서가 중요하다.
N 개 후보 중 K 개만 고르고 순서는 상관없다.	조합 백트래킹	선택 집합만 중요하다.
K 가 18 이고 방문 집합 상태가 반복된다.	비트마스킹 DP	$K!$ 대신 2^K 상태 압축을 검토한다.
최소 이동 횟수를 구한다.	BFS	같은 비용 최단거리다.
각 항목을 용량 내에서 선택한다.	배낭 DP	제한 자원과 가치 최적화다.

2. 개념

백트래킹은 하나를 선택하고 재귀 호출한 뒤 그 선택을 취소한다. 순열은 순서가 중요하고, 조합은 순서가 중요하지 않다.

순열에서는 visited 배열로 이미 선택한 원소를 표시한다. 조합에서는 다음 탐색 시작 인덱스를 넘겨 같은 조합의 순서만 바뀐 중복을 제거한다.

$K!$ 또는 2^N 의 크기를 대략 계산해야 한다. K가 8이면 $8!=40,320$ 이라 가능하지만 K가 15이면 순열은 거의 불가능하다.

3. 정답 선학습: 작은 체크포인트 순열

$cost[i][j]$ 는 i에서 j로 이동하는 비용이다. 0 번에서 시작해 나머지 모든 지점을 정확히 한 번씩 방문할 때의 최소 비용을 반환한다. 마지막 지점에서 0 번으로 돌아올 필요는 없다. 지점 수는 9 이하이다.

함수

```
public int solution(int[][] cost)
```

예시

$cost = \{\{0,2,9,8\},\{2,0,6,4\},\{9,6,0,3\},\{8,4,3,0\}\} \rightarrow 9$

완성 정답 코드

```
import java.util.*;

class Solution {
    private int[][] cost;
    private boolean[] visited;
    private int answer;

    public int solution(int[][] cost) {
        this.cost = cost;
        this.visited = new boolean[cost.length];
        this.answer = Integer.MAX_VALUE;
        visited[0] = true;
        dfs(0, 1, 0);
        return answer;
    }

    private void dfs(int current, int visitedCount, int totalCost) {
        if (totalCost >= answer) {
            return;
        }
        if (visitedCount == cost.length) {
            answer = Math.min(answer, totalCost);
            return;
        }
    }
}
```

```

        for (int next = 1; next < cost.length; next++) {
            if (visited[next]) {
                continue;
            }
            visited[next] = true;
            dfs(next, visitedCount + 1,
                totalCost + cost[current][next]);
            visited[next] = false;
        }
    }
}

```

4. 오늘 외울 코드 카드

1. 선택: visited[next]=true
2. 재귀 호출
3. 취소: visited[next]=false

빈칸 복원

```

for (int next = 1; next < n; next++) {
    if (visited[next]) continue;

    visited[next] = ____;
    dfs(next, count + 1, total + cost[current][next]);
    visited[next] = ____;
}

```

5. 프로그래머스형 변형 문제: 목표 합에 가까운 조합

values에서 정확히 k개를 선택해 합을 만든다. target과 선택 합의 절댓값 차이 중 최솟값을 반환하라. 선택 순서는 중요하지 않다.

제한사항

- 1 ≤ values.length ≤ 20
- 1 ≤ k ≤ values.length
- 값과 target은 int 범위

함수

```
public long solution(int[] values, int k, int target)
```

예시

values={8,3,5,12,7}, k=3, target=20 → 0

6. 단계별 힌트

힌트 1

조합이므로 start 인덱스를 재귀 인자로 둔다.

힌트 2

선택 개수가 k가 되면 차이를 갱신한다.

힌트 3

남은 원소 수가 필요한 선택 수보다 적으면 중단할 수 있다.

7. 변형 문제 정답

정답 코드

```

class Solution {
    private int[] values;
    private int k;
    private int target;
    private long answer;

    public long solution(int[] values, int k, int target) {
        this.values = values;
        this.k = k;
        this.target = target;
        this.answer = Long.MAX_VALUE;
        choose(0, 0, 0L);
    }
}

```

```

        return answer;
    }

    private void choose(int start, int count, long sum) {
        if (count == k) {
            answer = Math.min(answer, Math.abs(sum - target));
            return;
        }
        int need = k - count;
        if (values.length - start < need) {
            return;
        }

        for (int i = start; i < values.length; i++) {
            choose(i + 1, count + 1, sum + values[i]);
        }
    }
}

```

8. 오늘의 복습

- ☐ 순열과 조합의 차이를 한 문장으로 말할 수 있는가?
- ☐ 선택 취소가 빠지면 어떤 문제가 생기는가?
- ☐ K가 커질 때 비트마스크 DP를 고려해야 하는 이유는?

세 줄 기록

유형: _____ | 판단 단어: _____ | 외울 템플릿: _____

Day 11. 누적합·이분 탐색: 빠르게 합치고 답을 좁히기

예상 시간: 약 190 분

기준 백준: 11659 구간 합 구하기 4 · 2343 기타 레슨 · 3079 입국 심사

오늘 목표

- 구간 합을 $O(1)$ 에 계산한다.
- 정답 가능 여부가 단조일 때 이분 탐색을 적용한다.
- left와 right의 초기 범위를 설정한다.

유형 신호

구간 합 질문 반복 / 최소 가능한 최대값 / X로 가능하면 더 큰 값도 가능 / 정답 범위가 매우 큼

1. 유형 판별 연습

처음에는 정답을 가리지 않는다. 문장과 정답 유형을 함께 읽고, 판단 근거에 밑줄을 긋는다.

문장	정답 유형	판단 근거
수만 개의 $[l, r]$ 합 질문을 처리한다.	누적합	$prefix[r+1]-prefix[l]$ 로 계산한다.
최소 용량 X를 찾고 X가 커질수록 조건이 쉬워진다.	이분 탐색	가능 여부가 false에서 true로 한 번 바뀐다.
모든 경로를 깊게 탐색한다.	DFS	연결 탐색이다.
가중치 최소 경로를 구한다.	다익스트라	PriorityQueue와 dist를 쓴다.
정해진 용량에서 최대 가치를 구한다.	배낭 DP	용량별 최적값을 저장한다.

2. 개념

누적합 $prefix[i]$ 를 0번부터 $i-1$ 번까지의 합으로 정의하면 l 부터 r 까지의 합은 $prefix[r+1]-prefix[l]$ 이다. 한 번 $O(N)$ 으로 만들고 각 질문을 $O(1)$ 에 처리한다.

이분 탐색은 정렬된 배열에서 값 찾기뿐 아니라 정답 자체를 찾는 데도 쓴다. 어떤 용량 X로 작업이 가능하다면 더 큰 용량으로도 가능하다는 단조성이 있어야 한다.

연속 작업을 m 개 이하 그룹으로 나누며 그룹 최대 합을 최소화하는 문제에서 left는 원소 최댓값, right는 전체 합이다. mid 용량으로 필요한 그룹 수를 세어 가능 여부를 판단한다.

3. 정답 선학습: 구간 사용량 합

numbers와 $[l, r]$ 쿼리들이 주어진다. 각 쿼리 구간의 합을 $long[]$ 로 반환한다. 인덱스는 0부터 시작한다.

함수

```
public long[] solution(int[] numbers, int[][] queries)
```

예시

```
numbers={3,1,4,1,5}, queries={{0,2},{1,4},{3,3}} -> {8,11,1}
```

완성 정답 코드

```
class Solution {
    public long[] solution(int[] numbers, int[][] queries) {
        long[] prefix = new long[numbers.length + 1];
        for (int i = 0; i < numbers.length; i++) {
            prefix[i + 1] = prefix[i] + numbers[i];
        }

        long[] answer = new long[queries.length];
        for (int i = 0; i < queries.length; i++) {
            int left = queries[i][0];
            int right = queries[i][1];
            answer[i] = prefix[right + 1] - prefix[left];
        }
        return answer;
    }
}
```

4. 오늘 외울 코드 카드

1. prefix 크기는 N+1
2. $\text{prefix}[i+1] = \text{prefix}[i] + a[i]$
3. $\text{sum}(l, r) = \text{prefix}[r+1] - \text{prefix}[l]$

빈칸 복원

```
long[] prefix = new long[n + 1];
for (int i = 0; i < n; i++) {
    prefix[i + 1] = _____ + numbers[i];
}

long range = prefix[_____] - prefix[_____];
```

5. 프로그래머스형 변형 문제: 작업 묶음의 최소 최대 부하

순서가 고정된 lessons 를 연속한 m 개 이하의 묶음으로 나눈다. 각 묶음 합 중 최댓값을 가능한 한 작게 만들 때 그 최솟값을 반환하라. 원소 순서는 바꿀 수 없다.

제한사항

- 1 ≤ lessons.length ≤ 200,000
- 1 ≤ m ≤ lessons.length
- 1 ≤ lessons[i] ≤ 1,000,000,000

함수

```
public long solution(int[] lessons, int m)
```

예시

```
lessons={1,2,3,4,5,6,7,8,9}, m=3 → 17
```

6. 단계별 힌트

힌트 1

left 는 가장 큰 lesson, right 는 전체 합이다.

힌트 2

mid 를 한 묶음의 최대 허용 합으로 가정한다.

힌트 3

필요한 묶음 수가 m 이하이면 더 작은 답을 탐색한다.

7. 변형 문제 정답

정답 코드

```
class Solution {
    public long solution(int[] lessons, int m) {
        long left = 0L;
        long right = 0L;
        for (int lesson : lessons) {
            left = Math.max(left, lesson);
            right += lesson;
        }

        long answer = right;
        while (left <= right) {
            long mid = (left + right) >>> 1;
            if (canSplit(lessons, m, mid)) {
                answer = mid;
                right = mid - 1;
            } else {
                left = mid + 1;
            }
        }
        return answer;
    }

    private boolean canSplit(int[] lessons, int m, long limit) {
        int groups = 1;
        long sum = 0L;

        for (int lesson : lessons) {
```

```
        if (sum + lesson > limit) {
            groups++;
            sum = lesson;
            if (groups > m) {
                return false;
            }
        } else {
            sum += lesson;
        }
    }
    return true;
}
```

8. 오늘의 복습

- ☐ prefix[i]의 의미를 정확히 말할 수 있는가?
- ☐ 이분 탐색이 가능한 단조성은 무엇인가?
- ☐ left와 right를 왜 각각 최댓값과 전체 합으로 두는가?

세 줄 기록

유형: _____ | 판단 단어: _____ | 외울 템플릿: _____

Day 12. 비트마스크 BFS: 위치와 수집 상태를 함께 저장

예상 시간: 약 200 분

기준 백준: 17244 아 맞다 우산 · 1194 달이 차오른다, 가자.

오늘 목표

- 비트 연산의 기본을 익힌다.
- 같은 위치라도 보유 상태가 다르면 다른 방문 상태임을 이해한다.
- visited[r][c][mask] 구조를 사용한다.

유형 신호

수집해야 할 물건 K 개 / 보유한 열쇠·아이템 상태 / K 가 작음 / 위치 + 방문 집합

1. 유형 판별 연습

처음에는 정답을 가리지 않는다. 문장과 정답 유형을 함께 읽고, 판단 근거에 밑줄을 긋는다.

문장	정답 유형	판단 근거
격자에서 모든 아이템을 모은 뒤 출구로 간다.	비트마스크 BFS	위치와 수집 집합을 함께 방문 처리한다.
경유지 간 비용표에서 모든 지점을 방문한다.	비트마스크 DP	거리 계산보다 방문 순서 최적화가 핵심이다.
단순 연결 요소 개수다.	DFS/BFS	추가 보유 상태가 없다.
작업을 최대한 많이 선택한다.	그리디	정렬 기준으로 선택한다.
용량 제한에서 최대 가치를 구한다.	배낭 DP	용량별 최적값을 저장한다.

2. 개념

K 개의 아이템 방문 여부를 boolean[K] 대신 하나의 정수 mask 로 표현할 수 있다. i 번째 아이템을 보유했는지는 (mask & (1<<i)) != 0 으로 확인한다.

아이템을 새로 얻은 상태는 nextMask = mask | (1<<i)다. 모든 아이템을 모은 상태는 (1<<K)-1 이다.

같은 칸이라도 수집한 아이템이 다르면 이후 가능한 경로가 다르다. 따라서 visited[r][c]가 아니라 visited[r][c][mask]가 필요하다.

3. 정답 선학습: 모든 장비를 수집한 뒤 출구로

문자열 격자에서 S는 시작, E는 출구, I는 수집 장비, #은 벽, .은 빈 칸이다. 모든 I를 수집한 뒤 E에 도달하는 최소 이동 횟수를 반환한다. 불가능하면 -1 이다. I는 최대 6 개다.

함수

```
public int solution(String[] map)
```

예시

```
map={"S.I", ".#.", ".I.E"} -> 8
```

완성 정답 코드

```
import java.util.*;

class Solution {
    private static final int[] DR = {-1, 0, 1, 0};
    private static final int[] DC = {0, 1, 0, -1};

    public int solution(String[] map) {
        int rows = map.length;
        int cols = map[0].length();
        int[][] itemId = new int[rows][cols];
        for (int[] row : itemId) {
            Arrays.fill(row, -1);
        }

        int startR = 0;
        int startC = 0;
        int itemCount = 0;
        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                char cell = map[r].charAt(c);
                if (cell == 'S') {
                    startR = r;
                    startC = c;
                } else if (cell == 'I') {
                    itemCount++;
                }
            }
        }

        // BFS implementation would follow here
    }
}
```

```

        itemId[r][c] = itemCount++;
    }
}

int fullMask = (1 << itemCount) - 1;
boolean[][][] visited =
    new boolean[rows][cols][1 << itemCount];
ArrayDeque<int[]> queue = new ArrayDeque<>();
queue.offer(new int[]{startR, startC, 0, 0});
visited[startR][startC][0] = true;

while (!queue.isEmpty()) {
    int[] current = queue.poll();
    int r = current[0];
    int c = current[1];
    int mask = current[2];
    int distance = current[3];

    if (map[r].charAt(c) == 'E' && mask == fullMask) {
        return distance;
    }

    for (int d = 0; d < 4; d++) {
        int nr = r + DR[d];
        int nc = c + DC[d];
        if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) {
            continue;
        }
        if (map[nr].charAt(nc) == '#') {
            continue;
        }

        int nextMask = mask;
        if (itemId[nr][nc] != -1) {
            nextMask |= 1 << itemId[nr][nc];
        }
        if (visited[nr][nc][nextMask]) {
            continue;
        }

        visited[nr][nc][nextMask] = true;
        queue.offer(new int[]{nr, nc, nextMask, distance + 1});
    }
}
return -1;
}
}

```

4. 오늘 외울 코드 카드

1. i 번 비트 확인: $(mask \& (1 \ll i)) \neq 0$
2. i 번 비트 추가: $mask \mid (1 \ll i)$
3. 전체 방문 mask: $(1 \ll K) - 1$

빈칸 복원

```

int fullMask = (1 << itemCount) - 1;

int nextMask = mask;
if (itemId[nr][nc] != -1) {
    nextMask = mask _____ (1 << itemId[nr][nc]);
}

if (!visited[nr][nc][_____]) {
    visited[nr][nc][_____] = true;
}

```

5. 프로그래머스형 변형 문제: 진단 모듈 수집

garage 에서 S 는 시작, E 는 출구, C 는 진단 모듈, #은 벽이다. 모든 C 를 수집하고 E 에 도달하는 최소 이동 횟수를 반환하라. C 는 최대 7 개다.

제한사항

- 1 ≤ 행, 열 ≤ 50
- C ≤ 7
- S 와 E 는 하나씩 존재

함수

```
public int solution(String[] garage)
```

예시

```
garage={"S.C.", ".#..", "C..E"} -> 9
```

6. 단계별 힌트

힌트 1

모든 C 에 0 부터 번호를 붙인다.

힌트 2

큐 상태는 r,c,mask,distance 다.

힌트 3

E 에 도착해도 mask 가 fullMask 가 아니면 계속 탐색한다.

7. 변형 문제 정답

정답 코드

```
import java.util.*;

class Solution {
    private static final int[] DR = {-1, 0, 1, 0};
    private static final int[] DC = {0, 1, 0, -1};

    public int solution(String[] garage) {
        int rows = garage.length;
        int cols = garage[0].length();
        int[][] moduleId = new int[rows][cols];
        for (int[] row : moduleId) {
            Arrays.fill(row, -1);
        }

        int startR = -1;
        int startC = -1;
        int moduleCount = 0;
        for (int r = 0; r < rows; r++) {
            for (int c = 0; c < cols; c++) {
                char cell = garage[r].charAt(c);
                if (cell == 'S') {
                    startR = r;
                    startC = c;
                } else if (cell == 'C') {
                    moduleId[r][c] = moduleCount++;
                }
            }
        }

        int fullMask = (1 << moduleCount) - 1;
        boolean[][][] visited =
            new boolean[rows][cols][1 << moduleCount];
        ArrayDeque<int[]> queue = new ArrayDeque<>();
        queue.offer(new int[]{startR, startC, 0, 0});
        visited[startR][startC][0] = true;

        while (!queue.isEmpty()) {
            int[] current = queue.poll();
            int r = current[0];
            int c = current[1];
            int mask = current[2];
            int distance = current[3];

            if (garage[r].charAt(c) == 'E' && mask == fullMask) {
                return distance;
            }

            for (int d = 0; d < 4; d++) {
                int nr = r + DR[d];
                int nc = c + DC[d];
                if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) {
                    continue;
                }
                if (garage[nr].charAt(nc) == '#') {
                    continue;
                }

                int nextMask = mask;
                if (moduleId[nr][nc] != -1) {
                    nextMask |= 1 << moduleId[nr][nc];
                }
                if (visited[nr][nc][nextMask]) {
                    continue;
                }
                visited[nr][nc][nextMask] = true;
                queue.offer(new int[]{nr, nc, nextMask, distance + 1});
            }
        }
    }
}
```

```
    }  
    return -1;  
}  
}
```

8. 오늘의 복습

- ☐ 같은 칸을 여러 번 방문할 수 있는 이유는?
- ☐ mask 에 비트를 추가하는 연산은?
- ☐ K 가 커지면 visited 메모리가 어떻게 증가하는가?

세 줄 기록

유형: _____ | 판단 단어: _____ | 외울 템플릿: _____

Day 13. 다익스트라 전처리 + 비트마스크 DP

예상 시간: 약 240 분

기준 백준: 1504 특정한 최단 경로 · 4991 로봇 청소기 · 2098 외판원 순회

오늘 목표

- 중요 지점 간 최단비용을 전처리한다.
- $dp[mask][cur]$ 상태를 이해한다.
- 순열 $K!$ 을 $2^K K^2$ 으로 줄이는 구조를 익힌다.

유형 신호

필수 경유지 K 개 / 방문 순서는 자유 / 두 지점 사이 최단비용 / K 가 10~15 정도

1. 유형 판별 연습

처음에는 정답을 가리지 않는다. 문장과 정답 유형을 함께 읽고, 판단 근거에 밑줄을 긋는다.

문장	정답 유형	판단 근거
격자에서 경유지 12 개를 모두 방문해야 한다.	최단거리 전처리+비트마스크 DP	순열 12!보다 2^{12} 상태가 적다.
경유지가 3 개뿐이다.	전처리+순열도 가능	$K!$ 이 충분히 작다.
한 번 이동 비용이 모두 1 인 격자다.	BFS 전처리	지점 간 거리는 BFS로 구할 수 있다.
칸 비용이 서로 다르다.	다익스트라 전처리	가중치가 다르다.
방문 집합과 현재 위치가 같으면 이후 선택이 같다.	비트마스크 DP 가능	과거 순서를 상태에서 제거할 수 있다.

2. 개념

큰 격자 전체에서 경유지 순서를 함께 탐색하면 상태가 너무 커진다. 시작점, 경유지, 도착점만 중요한 지점으로 뽑고 각 지점에서 다익스트라를 실행해 $pairCost[i][j]$ 를 만든다.

그 뒤 격자를 잊고 작은 완전 그래프에서 방문 순서만 최적화한다. $dp[mask][cur]$ 는 $mask$ 에 해당하는 경유지를 방문했고 현재 cur 경유지에 있을 때의 최소 비용이다.

같은 $mask$ 와 cur 에 여러 순서가 도달할 수 있지만 앞으로 가능한 선택은 같다. 따라서 그중 최소 비용 하나만 남긴다. 이것이 순열을 DP로 압축하는 핵심이다.

3. 정답 선학습: 비용 행렬의 필수 지점 방문

$cost$ 는 시작점 0, 필수 지점 1..N-2, 도착점 N-1 사이의 이동 비용 행렬이다. -1은 이동 불가다. 모든 필수 지점을 한 번씩 방문하고 도착점에 가는 최소 비용을 반환한다. 불가능하면 -1이다.

함수

```
public long solution(long[][] cost)
```

예시

```
cost={{0,4,2,10},{4,0,1,5},{2,1,0,8},{10,5,8,0}} -> 8
```

완성 정답 코드

```
import java.util.*;

class Solution {
    private static final long INF = Long.MAX_VALUE / 4;

    public long solution(long[][] cost) {
        int pointCount = cost.length;
        int k = pointCount - 2;
        int end = pointCount - 1;

        if (k == 0) {
            return cost[0][end];
        }

        long[][] dp = new long[1 << k][k];
        for (long[] row : dp) {
            Arrays.fill(row, INF);
        }
    }
}
```



```

    for (int i = 0; i < k; i++) {
        if (cost[0][i + 1] != -1) {
            dp[1 << i][i] = cost[0][i + 1];
        }
    }

    for (int mask = 1; mask < (1 << k); mask++) {
        for (int current = 0; current < k; current++) {
            if (dp[mask][current] == INF) {
                continue;
            }
            for (int next = 0; next < k; next++) {
                if ((mask & (1 << next)) != 0) {
                    continue;
                }
                long move = cost[current + 1][next + 1];
                if (move == -1) {
                    continue;
                }

                int nextMask = mask | (1 << next);
                dp[nextMask][next] = Math.min(
                    dp[nextMask][next],
                    dp[mask][current] + move
                );
            }
        }
    }

    int fullMask = (1 << k) - 1;
    long answer = INF;
    for (int last = 0; last < k; last++) {
        if (cost[last + 1][end] == -1) {
            continue;
        }
        answer = Math.min(answer,
            dp[fullMask][last] + cost[last + 1][end]);
    }
    return answer == INF ? -1 : answer;
}

```

4. 오늘 외울 코드 카드

1. dp[mask][cur] = 방문 집합 mask, 현재 경유지 cur 의 최소 비용
2. nextMask = mask | (1 << next)
3. 모든 경유지 방문 후 마지막 경유지에서 도착점 비용을 더한다.

빈칸 복원

```

int nextMask = mask | (1 << next);
dp[nextMask][next] = Math.min(
    dp[nextMask][next],
    dp[mask][current] + cost[current + 1][next + 1]
);

```

5. 프로그래머스형 변형 문제: 필수 점검소가 있는 격자 최소비용

board 의 -1 은 벽, 양수는 해당 칸에 진입할 때의 비용이다. start 에서 출발해 stops 를 모두 임의 순서로 방문하고 end 에 도착하는 최소 비용을 반환하라. 시작 칸 비용도 한 번 포함하며, 불가능하면 -1 을 반환한다.

제한사항

- 1 ≤ 행, 열 ≤ 100
- 0 ≤ stops.length ≤ 12
- 이동 가능 칸 비용 ≤ 1,000,000
- start, end, stops 는 서로 다른 이동 가능 칸

함수

```
public long solution(int[][] board, int[] start, int[] end, int[][] stops)
```

예시

```
board={{1,3,1,9,1},{1,1,1,9,1},{9,9,1,1,1},{1,1,1,9,1}}, start={0,0}, end={3,4}, stops={{0,2},{2,3}} -> 10
```

6. 단계별 힌트

힌트 1

points=[start, stops..., end]를 만든다.

힌트 2

각 point 에서 다익스트라를 실행해 pairCost 를 만든다.

힌트 3

전처리 dist[source]=0 으로 두면 출발 칸 비용을 제외하고 도착 칸 비용을 포함한다. 마지막 경답에 board[start]를 한 번 더한다.

7. 변형 문제 정답

정답 코드

```
import java.util.*;

class Solution {
    private static final long INF = Long.MAX_VALUE / 4;
    private static final int[] DR = {-1, 0, 1, 0};
    private static final int[] DC = {0, 1, 0, -1};

    private static class Node implements Comparable<Node> {
        int r;
        int c;
        long cost;

        Node(int r, int c, long cost) {
            this.r = r;
            this.c = c;
            this.cost = cost;
        }

        public int compareTo(Node other) {
            return Long.compare(cost, other.cost);
        }
    }

    public long solution(int[][] board, int[] start,
                        int[] end, int[][] stops) {
        int k = stops.length;
        int[][] points = new int[k + 2][2];
        points[0] = start.clone();
        for (int i = 0; i < k; i++) {
            points[i + 1] = stops[i].clone();
        }
        points[k + 1] = end.clone();

        long[][] pairCost = new long[k + 2][k + 2];
        for (int i = 0; i < k + 2; i++) {
            long[][] dist = dijkstra(board, points[i]);
            for (int j = 0; j < k + 2; j++) {
                pairCost[i][j] = dist[points[j][0]][points[j][1]];
            }
        }

        if (k == 0) {
            long move = pairCost[0][1];
            return move == INF ? -1L : board[start[0]][start[1]] + move;
        }

        long[][] dp = new long[1 << k][k];
        for (long[] row : dp) {
            Arrays.fill(row, INF);
        }

        for (int i = 0; i < k; i++) {
            if (pairCost[0][i + 1] != INF) {
                dp[1 << i][i] = pairCost[0][i + 1];
            }
        }

        for (int mask = 1; mask < (1 << k); mask++) {
            for (int current = 0; current < k; current++) {
                if (dp[mask][current] == INF) {
                    continue;
                }
                for (int next = 0; next < k; next++) {
                    if ((mask & (1 << next)) != 0) {
                        continue;
                    }
                    long move = pairCost[current + 1][next + 1];
                    if (move == INF) {
                        continue;
                    }
                    int nextMask = mask | (1 << next);
                    dp[nextMask][next] = Math.min(
                        dp[nextMask][next],
                        dp[mask][current] + move
                    );
                }
            }
        }
    }
}
```

```

int fullMask = (1 << k) - 1;
long answer = INF;
for (int last = 0; last < k; last++) {
    long moveToEnd = pairCost[last + 1][k + 1];
    if (moveToEnd == INF) {
        continue;
    }
    answer = Math.min(answer,
        dp[fullMask][last] + moveToEnd);
}

if (answer == INF) {
    return -1L;
}
return board[start[0]][start[1]] + answer;
}

private long[][] dijkstra(int[][] board, int[] source) {
    int rows = board.length;
    int cols = board[0].length;
    long[][] dist = new long[rows][cols];
    for (long[] row : dist) {
        Arrays.fill(row, INF);
    }

    PriorityQueue<Node> pq = new PriorityQueue<>();
    dist[source[0]][source[1]] = 0L;
    pq.offer(new Node(source[0], source[1], 0L));

    while (!pq.isEmpty()) {
        Node current = pq.poll();
        if (current.cost != dist[current.r][current.c]) {
            continue;
        }

        for (int d = 0; d < 4; d++) {
            int nr = current.r + DR[d];
            int nc = current.c + DC[d];
            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) {
                continue;
            }
            if (board[nr][nc] == -1) {
                continue;
            }

            long nextCost = current.cost + board[nr][nc];
            if (nextCost < dist[nr][nc]) {
                dist[nr][nc] = nextCost;
                pq.offer(new Node(nr, nc, nextCost));
            }
        }
    }
    return dist;
}
}

```

8. 오늘의 복습

- ☐ 왜 격자를 중요한 지점 그래프로 압축하는가?
- ☐ dp 상태에 방문 집합뿐 아니라 현재 위치가 필요한 이유는?
- ☐ 출발 칸 비용 중복을 어떻게 방지했는가?

세 줄 기록

유형: _____ | 판단 단어: _____ | 외울 템플릿: _____

Day 14. 최종 모의고사: 그리디 + 최단경로·비트마스크 DP

예상 시간: 총 180 분

기준 백준: 1744 수 묶기 · 1504 특정한 최단 경로 · 2098 외판원 순회

오늘 목표

- 문제 A는 50 분, 문제 B는 130 분으로 나눠 푼다.
- 유형 이름과 템플릿을 먼저 적는다.
- 풀이 후 코드가 아니라 판단 과정부터 복기한다.

유형 신호

문제 A: 양수·음수 분리와 정렬 / 문제 B: 필수 경유지와 최단거리 전처리

1. 유형 판별 연습

처음에는 정답을 가리지 않는다. 문장과 정답 유형을 함께 읽고, 판단 근거에 밑줄을 긋는다.

문장	정답 유형	판단 근거
처리 순서를 자유롭게 바꿀 수 있고 양수·음수 효과가 있다.	그리디 후보	부호 분리와 정렬 순서를 검토한다.
경유지 12 개의 방문 순서를 정해야 한다.	비트마스크 DP	순열보다 방문 집합 상태가 적다.
지점 사이 도로 비용이 다르다.	다익스트라	양의 가중치 최단거리다.
같은 방문 집합과 현재 위치에 여러 경로가 도착한다.	DP로 최소값만 보존	이후 선택이 같기 때문이다.
정답이 int 범위를 넘을 수 있다.	long 사용	비용 합과 INF 계산을 안전하게 한다.

2. 개념

모의고사에서는 처음 5 분 동안 문제별 유형 후보를 적는다. 문제 A는 정렬과 누적 상태, 문제 B는 다익스트라 전처리와 비트마스크 DP다.

문제 A는 완성된 정렬 순서가 왜 가능한지 작은 교환으로 확인한다. 양수 적은 먼저 처리해 에너지를 늘리고, 양수 내부에서는 작은 값부터 처리하는 것이 문턱 조건을 낮춘다.

문제 B는 큰 그래프를 중요한 지점만 남긴 작은 그래프로 바꾼다. 각 지점에서 다익스트라를 한 번씩 수행한 뒤 Day 13의 `dp[mask][cur]`를 그대로 적용한다.

3. 정답 선학습: 문제 A - 최소 초기 에너지

0 이 아닌 정수 `enemies`가 주어진다. 현재 에너지가 `current`일 때 `enemy`를 처치하려면 `current > enemy`여야 한다. 처치 후 `current += enemy`가 되며 `current <= 0`이면 실패한다. 순서를 자유롭게 정할 때 모든 적을 처치할 수 있는 최소 양의 초기 에너지를 반환하라.

함수

```
public long solution(int[] enemies)
```

예시

```
enemies={10,-100,5,-20,100} -> 86
```

완성 정답 코드

```
import java.util.*;

class Solution {
    public long solution(int[] enemies) {
        long total = 0L;
        List<Integer> positives = new ArrayList<>();

        for (int enemy : enemies) {
            total += enemy;
            if (enemy > 0) {
                positives.add(enemy);
            }
        }
        Collections.sort(positives);

        long answer = Math.max(1L, 1L - total);
    }
}
```

```

        long positivePrefix = 0L;

        for (int enemy : positives) {
            answer = Math.max(
                answer,
                enemy + 1L - positivePrefix
            );
            positivePrefix += enemy;
        }
        return answer;
    }
}

```

4. 오늘 외울 코드 카드

1. 양수는 오름차순으로 먼저 처리
2. 양수 enemy 전 조건: $\text{initial} \geq \text{enemy} + 1 - \text{prefix}$
3. 전체 종료 조건: $\text{initial} \geq 1 - \text{total}$

빈칸 복원

```

long answer = Math.max(1L, 1L - total);
long prefix = 0L;

for (int enemy : positives) {
    answer = Math.max(answer, enemy + 1L - _____);
    prefix += _____;
}

```

5. 프로그래머스형 변형 문제: 문제 B - 필수 감사 지점이 있는 도로망

정점 $0..n-1$ 의 무방향 도로망이 있다. $\text{roads}[i]=[u,v,\text{cost}]$ 이며 cost 는 양수다. start 에서 출발해 checkpoints 를 모두 임의 순서로 방문하고 end 에 도착하는 최소 비용을 반환하라. 불가능하면 -1 이다.

제한사항

- $2 \leq n \leq 5,000$
- $1 \leq \text{roads.length} \leq 20,000$
- $0 \leq \text{checkpoints.length} \leq 12$
- 모든 비용은 양수

함수

```
public long solution(int n, int[][] roads, int start, int end, int[] checkpoints)
```

예시

$n=6$, $\text{roads}=\{\{0,1,2\},\{0,2,5\},\{1,2,1\},\{1,3,4\},\{2,3,1\},\{3,4,3\},\{2,5,7\},\{4,5,1\}\}$, $\text{start}=0$, $\text{end}=5$, $\text{checkpoints}=\{2,4\}$ -> 8

6. 단계별 힌트

힌트 1

$\text{points}=[\text{start}, \text{checkpoints}..., \text{end}]$ 를 만든다.

힌트 2

각 point 에서 그래프 다익스트라를 실행한다.

힌트 3

pairCost 가 완성되면 Day 13 의 비트마스크 DP 와 동일하다.

7. 변형 문제 정답

정답 코드

```

import java.util.*;

class Solution {
    private static final long INF = Long.MAX_VALUE / 4;

    private static class Edge {
        int to;
        int cost;
    }
}

```

```

    Edge(int to, int cost) {
        this.to = to;
        this.cost = cost;
    }
}

private static class Node implements Comparable<Node> {
    int vertex;
    long cost;

    Node(int vertex, long cost) {
        this.vertex = vertex;
        this.cost = cost;
    }

    public int compareTo(Node other) {
        return Long.compare(cost, other.cost);
    }
}

public long solution(int n, int[][] roads, int start,
                    int end, int[] checkpoints) {
    List<Edge>[] graph = new ArrayList[n];
    for (int i = 0; i < n; i++) {
        graph[i] = new ArrayList<>();
    }
    for (int[] road : roads) {
        graph[road[0]].add(new Edge(road[1], road[2]));
        graph[road[1]].add(new Edge(road[0], road[2]));
    }

    int k = checkpoints.length;
    int[] points = new int[k + 2];
    points[0] = start;
    for (int i = 0; i < k; i++) {
        points[i + 1] = checkpoints[i];
    }
    points[k + 1] = end;

    long[][] pairCost = new long[k + 2][k + 2];
    for (int i = 0; i < k + 2; i++) {
        long[] dist = dijkstra(graph, points[i]);
        for (int j = 0; j < k + 2; j++) {
            pairCost[i][j] = dist[points[j]];
        }
    }

    if (k == 0) {
        return pairCost[0][1] == INF ? -1L : pairCost[0][1];
    }

    long[][] dp = new long[1 << k][k];
    for (long[] row : dp) {
        Arrays.fill(row, INF);
    }
    for (int i = 0; i < k; i++) {
        dp[1 << i][i] = pairCost[0][i + 1];
    }

    for (int mask = 1; mask < (1 << k); mask++) {
        for (int current = 0; current < k; current++) {
            if (dp[mask][current] == INF) {
                continue;
            }
            for (int next = 0; next < k; next++) {
                if ((mask & (1 << next)) != 0) {
                    continue;
                }
                long move = pairCost[current + 1][next + 1];
                if (move == INF) {
                    continue;
                }
                int nextMask = mask | (1 << next);
                dp[nextMask][next] = Math.min(
                    dp[nextMask][next],
                    dp[mask][current] + move
                );
            }
        }
    }

    long answer = INF;
    int fullMask = (1 << k) - 1;
    for (int last = 0; last < k; last++) {
        long moveToEnd = pairCost[last + 1][k + 1];
        if (moveToEnd == INF) {
            continue;
        }
        answer = Math.min(answer,
            dp[fullMask][last] + moveToEnd);
    }
    return answer == INF ? -1L : answer;
}

```

```

private long[] dijkstra(List<Edge>[] graph, int source) {
    long[] dist = new long[graph.length];
    Arrays.fill(dist, INF);
    PriorityQueue<Node> pq = new PriorityQueue<>();
    dist[source] = 0L;
    pq.offer(new Node(source, 0L));

    while (!pq.isEmpty()) {
        Node current = pq.poll();
        if (current.cost != dist[current.vertex]) {
            continue;
        }
        for (Edge edge : graph[current.vertex]) {
            long nextCost = current.cost + edge.cost;
            if (nextCost < dist[edge.to]) {
                dist[edge.to] = nextCost;
                pq.offer(new Node(edge.to, nextCost));
            }
        }
    }
    return dist;
}

```

8. 오늘의 복습

- ☐ 문제 A의 정렬 순서를 말할 수 있는가?
- ☐ 문제 B에서 다익스트라는 무엇을 계산하고 DP는 무엇을 계산하는가?
- ☐ 모의고사에서 가장 오래 걸린 단계는 유형 판별, 상태 설계, 구현 중 무엇인가?

세 줄 기록

유형: _____ | 판단 단어: _____ | 외울 템플릿: _____

부록 A. 14 일 후 최소 암기 템플릿

Java 공통 제출 형태

```
import java.util.*; / class Solution { public 반환형 solution(매개변수) { ... } }
```

- • 배열 합과 최단 비용은 long 가능성을 먼저 확인한다.
- • Queue 는 ArrayDeque, 최단 비용 후보는 PriorityQueue 를 우선 사용한다.
- • 비교 함수는 뺄셈 대신 Integer.compare 또는 Long.compare 를 사용한다.

유형	암기 문장
격자 방향	int[] dr={-1,0,1,0}; int[] dc={0,1,0,-1};
BFS	Queue + dist=-1 + offer 시 방문 처리
DFS	visited=true + 4 방향 재귀 + 크기 누적
그리디	정렬 기준 -> 앞에서부터 선택 -> 상태 갱신
우선순위 큐	시작 정렬 + 종료된 항목 poll + 새 종료 offer
다익스트라	dist=INF + PQ + relax + 오래된 항목 제거
기본 DP	상태 한 문장 + 초기값 + 점화식 + 답 위치
배낭	for item -> for capacity descending
백트래킹	선택 -> 재귀 -> 선택 취소
비트마스크	확인 &: 추가 : 전체 (1<<K)-1
비트마스크 DP	dp[mask][cur] -> nextMask, next 갱신

부록 B. 학습 기록표

Day	완성 코드 입력	빈칸 복원	변형 시도	정답 비교	다음 복습일
1	☐	☐	☐	☐	_____
2	☐	☐	☐	☐	_____
3	☐	☐	☐	☐	_____
4	☐	☐	☐	☐	_____
5	☐	☐	☐	☐	_____
6	☐	☐	☐	☐	_____
7	☐	☐	☐	☐	_____
8	☐	☐	☐	☐	_____
9	☐	☐	☐	☐	_____
10	☐	☐	☐	☐	_____
11	☐	☐	☐	☐	_____
12	☐	☐	☐	☐	_____
13	☐	☐	☐	☐	_____
14	☐	☐	☐	☐	_____